Gap-Aware Retrieval (GAR)

A Gap-Aware Retrieval Algorithm for Iterative Retrieval-Augmented Generation

Abstract

Retrieval systems commonly rank documents according to their relevance to the original query. This can be insufficient in iterative information-seeking tasks, where the system already has some evidence and needs to retrieve only the information that is still missing. This project proposes **Gap-Aware Retrieval (GAR)**, a retrieval algorithm that uses three signals: relevance to the question and information gap, coverage of the missing information, and novelty with respect to previously collected evidence.

GAR is evaluated inside the existing MIGRES framework, which identifies missing information during iterative retrieval. MIGRES is the experimental framework; GAR is the proposed retrieval component. The surrounding MIGRES components are kept fixed as far as the original implementation permits. GAR is compared with BM25, DPR, Contriever, ColBERTv2, and SPLADE on the datasets used in the MIGRES evaluation: WikiHop, HotpotQA, MuSiQue, Natural Questions, TriviaQA, and StrategyQA.

The study evaluates retrieval quality, downstream answer quality, ablations, efficiency, and statistical significance. GAR has no trainable parameters in the proposed implementation. Its scoring hyperparameters are selected on a held-out validation set using Bayesian optimization and then frozen before test evaluation. The formulation is motivated by prior work on missing-information-guided retrieval, iterative evidence coverage, relevant information gain, sentence embeddings, and efficient nearest-neighbor search [Wang et al., 2025, Yadav et al., 2020, Pickett et al., 2024, Reimers and Gurevych, 2019, Johnson et al., 2017].

1 Introduction

Retrieval-Augmented Generation (RAG) systems use external documents to provide a language model with information that may not be available in its parameters. A retrieval algorithm therefore has a simple but important job: given a question and a document collection, return a small set of useful documents.

A common retrieval objective is query–document relevance:

$$q + D \longrightarrow d_1, d_2, \ldots, d_k.$$

This works well when the question can be answered from one retrieval step. It becomes less direct when the system retrieves information iteratively. After one step, some parts of the information need may already be satisfied. The next retrieval step should focus on the remaining gap rather than repeatedly retrieving documents about the same topic.

MIGRES provides a useful setting for this problem. MIGRES is a Missing Information Guided Retrieve-Extraction-Solving framework. It identifies missing information after evidence is collected and uses that missing information to guide subsequent retrieval [Wang et al., 2025]. In this project, we do not modify the MIGRES reasoning, extraction, or solving framework. Instead, we replace its retrieval component with GAR.

The central research question is:

Does explicitly modeling the current information gap improve retrieval?

GAR addresses this by scoring a candidate document using:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q, G_t) + \beta C(d, G_t) + \gamma N(d, E_t)}$$

where R measures relevance, C measures gap coverage, and N measures evidence novelty.

The contribution of this project is the proposed GAR retrieval formulation and its controlled empirical evaluation. The individual ideas used to construct GAR are grounded in previous work; the project does not claim that semantic relevance, information coverage, or novelty are individually new.

1.1 What is MIGRES and How Does It Work?

MIGRES stands for *Missing Information Guided Retrieve-Extraction-Solving*. It is an iterative framework designed for questions that require multiple pieces of information. Instead of retrieving documents only once, MIGRES repeatedly checks what information is already available, identifies what is still missing, retrieves information for that gap, and uses the retrieved information to continue solving the question [Wang et al., 2025].

The basic idea is:

$$\boxed{\text{Question} \rightarrow \text{Find Missing Information} \rightarrow \text{Retrieve} \rightarrow \text{Extract} \rightarrow \text{Update Information} \rightarrow \text{Repeat if Needed}}$$

In simple terms, MIGRES asks three questions during the process:

1. **What do we already know?** It keeps the evidence collected so far.
2. **What information is still missing?** It identifies the missing information needed to answer the question.
3. **What should we retrieve next?** It uses the missing information to guide the next retrieval step.

For example, suppose a question requires two facts, A and B. After the first retrieval, MIGRES may obtain A but determine that B is still missing. It then focuses the next retrieval step on B:

$$\text{Question} \rightarrow \text{retrieve } A \rightarrow \text{missing } B \rightarrow \text{retrieve } B \rightarrow \text{answer.}$$

MIGRES therefore maintains an iterative state consisting of the original question q, previously collected evidence E_t, and the current missing-information requirements G_t. These are provided to the retrieval component at each iteration:

$$\boxed{(q, E_t, G_t, D)}$$

where D is the searchable document corpus.

In our work, **MIGRES is the surrounding framework, not our contribution**. We use its iterative missing-information and evidence-generation process, while replacing its retrieval component with our proposed GAR algorithm. MIGRES determines *what information is missing*; GAR determines *which documents are most useful for obtaining that information.*

Thus, the division of responsibility is:

$$\boxed{\underbrace{\text{MIGRES}}_{\text{finds what is missing}} \rightarrow \underbrace{\text{GAR}}_{\text{retrieves useful documents}} \rightarrow \underbrace{\text{MIGRES}}_{\text{extracts and updates evidence}}}$$

This allows us to test whether a gap-aware retrieval algorithm improves the retrieval stage while keeping the rest of the MIGRES framework unchanged.

1.2 What We Use from MIGRES

We use MIGRES as the surrounding experimental framework. Specifically, GAR receives the state produced by MIGRES:

$$(q, E_t, G_t, D),$$

where:

- q is the original question;
- E_t is evidence collected before retrieval iteration t;
- G_t is the current missing-information state;
- D is the searchable document corpus.

GAR returns a ranked list of documents:

$$T_t = \mathrm{TopK}(\mathcal{C}_t).$$

MIGRES then continues its normal filtering, extraction, gap-analysis, and solving process.

We therefore compare:

$$\boxed{\text{MIGRES+BM25}} \quad \text{vs.} \quad \boxed{\text{MIGRES+GAR}}.$$

We additionally compare GAR with four other established retrieval methods.

1.3 What We Do Not Change

The following MIGRES components should remain fixed wherever the original implementation permits:

- question processing;
- information-gap generation;
- query-generation logic;
- knowledge filtering;
- information extraction;
- answer-generation/solving model;
- stopping conditions;
- prompts and model settings;
- dataset and corpus.

The main experimental variable is therefore the retrieval algorithm.

2 Information Retrieval Algorithms

2.1 What is an Information Retrieval Algorithm?

An *Information Retrieval (IR) algorithm* is a method used to search a collection of documents and rank the documents according to how useful they are for a given information need. In our setting, the information need is represented by a question or retrieval query q, and the IR algorithm produces an ordered list of documents:

$$\boxed{(q, D) \rightarrow d_1, d_2, \ldots, d_k}$$

where D is the document collection and $d_1, \ldots, d_k$ are the top-k retrieved documents.

Different IR algorithms use different signals to decide which documents should be ranked highly. For example, BM25 primarily uses lexical term-matching, DPR and Contriever use dense vector representations, ColBERTv2 uses token-level late interaction, and SPLADE uses sparse neural representations [Robertson and Zaragoza, 2009, Karpukhin et al., 2020, Izacard et al., 2022, Santhanam et al., 2022, Formal et al., 2021].

In a conventional retrieval system, the main objective is:

$$\boxed{\text{Find documents that are relevant to the query.}}$$

However, in an iterative system such as MIGRES, relevance to the original question may not be sufficient. Some information may already have been retrieved, while other information is still missing. This motivates GAR, which additionally considers the current information gap and the evidence already collected.

Thus, the difference between the baseline retrieval algorithms and GAR is not that GAR is simply another embedding model. GAR is a complete retrieval and ranking method whose scoring function explicitly incorporates the current information state:

$$\boxed{\text{Question} + \text{Information Gap} + \text{Existing Evidence} \rightarrow \text{Document Ranking.}}$$

2.2 Why Compare Retrieval Algorithms?

GAR is a retrieval algorithm, so the correct comparison is against established retrieval algorithms rather than against embedding models. The primary systems are:

$$\boxed{\text{BM25, DPR, Contriever, ColBERTv2, SPLADE, GAR}}$$

The five established methods are used through their published or official implementations where possible. GAR is implemented directly from the proposed formulation.

2.3 BM25

BM25 is a sparse probabilistic retrieval method based mainly on query-term frequency, inverse document frequency, and document-length normalization [Robertson and Zaragoza, 2009].

A common form is:

$$S_{\text{BM25}}(q, d) = \sum_{t \in q} IDF(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{avgdl}\right)}.$$

Here $f(t, d)$ is the frequency of term t in document d, $|d|$ is document length, $avgdl$ is average document length, and k_1, b are BM25 parameters.

BM25 is especially important because it is the retrieval baseline used in the MIGRES setting.

2.4 DPR

Dense Passage Retrieval (DPR) uses separate dense encoders for questions and passages and retrieves documents according to their vector similarity [Karpukhin et al., 2020].

A simplified score is:

$$S_{\text{DPR}}(q, d) = \mathbf{q}^T \mathbf{d},$$

where $\mathbf{q}$ and $\mathbf{d}$ are dense query and document representations.

DPR provides a supervised dense-retrieval baseline.

2.5 Contriever

Contriever is an unsupervised dense retrieval method based on contrastive learning [Izacard et al., 2022]. Its basic retrieval operation can be written as:

$$S_{\text{Contriever}}(q, d) = sim(\mathbf{q}, \mathbf{d}).$$

Contriever provides a dense-retrieval baseline with a different training approach from DPR.

2.6 ColBERTv2

ColBERTv2 uses late interaction. Instead of representing a complete document with only one vector, it keeps token-level representations and computes efficient token-level interactions [Santhanam et al., 2022].

A simplified score is:

$$S_{\text{ColBERT}}(q, d) = \sum_{i \in q} \max_{j \in d} sim(\mathbf{q}_i, \mathbf{d}_j).$$

This gives a useful comparison with both single-vector dense retrieval and sparse retrieval.

2.7 SPLADE

SPLADE is a neural sparse retrieval method that produces sparse vocabulary representations for queries and documents [Formal et al., 2021].

Let:

$$\mathbf{s}_d = f_\theta(d)$$

be a sparse document representation. A simplified retrieval score is:

$$S_{\text{SPLADE}}(q, d) = \mathbf{s}_q^T \mathbf{s}_d.$$

SPLADE provides a neural sparse baseline.

2.8 Baseline Comparison

System	Retrieval type	Role
MIGRES+BM25	Sparse lexical	Main baseline
MIGRES+DPR	Dense bi-encoder	Dense baseline
MIGRES+Contriever	Unsupervised dense	Dense baseline
MIGRES+ColBERTv2	Late interaction	Multi-vector baseline
MIGRES+SPLADE	Neural sparse	Neural sparse baseline
MIGRES+GAR	Gap-aware retrieval	Proposed method

All baselines should use established implementations and published checkpoints where possible. Their exact checkpoint, index, and retrieval parameters must be recorded.

3 Development and Introduction of GAR

3.1 What GAR Is

GAR is a retrieval algorithm designed for iterative retrieval. It changes the retrieval question from:

"Is this document relevant to the original question?"

to:

"Is this document useful given what is missing and what is already known?"

GAR uses three signals:

1. **Gap relevance**: relevance to the original question and the current information gap.
2. **Gap coverage**: how much of the missing information is covered by the candidate.
3. **Evidence novelty**: how much new information the candidate adds beyond existing evidence.

3.2 Why GAR Was Developed

A query-only retriever can repeatedly return documents that are strongly related to the question but do not resolve the current missing information. MIGRES already exposes the missing-information state, making it possible for the retrieval algorithm to use that state directly [Wang et al., 2025].

The motivation for the GAR components comes from three sources.

First, MIGRES provides the missing-information concept: later retrieval can be targeted toward information that is still missing [Wang et al., 2025].

Second, Yadav et al. use an iterative evidence-retrieval process that focuses later retrieval on information not covered by existing justifications. This motivates the coverage component in GAR [Yadav et al., 2020].

Third, Relevant Information Gain argues that retrieval should avoid wasting limited context on redundant information while retaining relevance [Pickett et al., 2024]. This motivates the novelty component.

GAR combines these ideas into one retrieval score. The formulation is therefore a proposed unified model; the project does not claim that each component is independently novel.

3.3 Applications

GAR is intended for settings where information needs change during retrieval, including:

- multi-hop question answering;
- iterative RAG;
- research-oriented evidence retrieval;
- evidence gathering over large document collections;
- systems that explicitly track missing information.

GAR does not require a new language model. The surrounding system can provide the current gap and evidence state.

4 Mathematical Formulation

4.1 Notation

At retrieval iteration t:

$$q = \text{original question},$$

$$E_t = \text{evidence collected before iteration } t,$$

$$G_t = \{g_1, \ldots, g_m\}$$

is the set of current information requirements, and

$$D = \text{searchable document corpus}.$$

A candidate document is:

$$d \in D.$$

An individual evidence item is:

$$e \in E_t.$$

The GAR input is therefore:

$$\boxed{(q, E_t, G_t, D)}.$$

4.2 Text Embeddings

GAR needs a fixed representation function:

$$\phi : \mathcal{T} \to \mathbb{R}^p.$$

We use:

`sentence-transformers/multi-qa-mpnet-base-cos-v1`

as the fixed embedding model. The model maps sentences and paragraphs to 768-dimensional dense vectors and was designed for semantic search [Reimers and Gurevych, 2019, Sentence Transformers, 2020].

The same model is used for questions, gaps, documents, and evidence. Document embeddings are computed once and stored in the ANN index.

4.3 Cosine Similarity

For two text items x and y:

$$\cos(x, y) = \frac{\phi(x)^T \phi(y)}{\|\phi(x)\|_2 \|\phi(y)\|_2}.$$

For a normalized score in $[0, 1]$, GAR uses:

$$\boxed{s(x, y) = \frac{1}{2}\left(1 + \frac{\phi(x)^T \phi(y)}{\|\phi(x)\|_2 \|\phi(y)\|_2}\right)} \tag{1}$$

The embedding model creates vectors; cosine similarity compares them.

4.4 Gap Weights

Each gap requirement has a non-negative weight:

$$w_i \geq 0, \qquad \sum_{i=1}^{m} w_i > 0.$$

For the first implementation:

$$\boxed{w_i = 1 \quad \forall i.}$$

This avoids introducing another learned model.

4.5 Gap Relevance

First calculate the weighted semantic relevance to the gap:

$$\boxed{s_G(d, G_t) = \frac{\sum_{i=1}^{m} w_i \, s(d, g_i)}{\sum_{i=1}^{m} w_i}} \tag{2}$$

Then combine original-question relevance and gap relevance:

$$\boxed{R(d, q, G_t) = \lambda s(d, q) + (1 - \lambda) s_G(d, G_t)} \tag{3}$$

where:

$$0 \leq \lambda \leq 1.$$

Interpretation:

- λ close to 1: emphasize the original question;
- λ close to 0: emphasize the information gap.

4.6 Gap Coverage

Let $\tau(x)$ be the set of normalized content tokens obtained after the fixed preprocessing pipeline.

For a non-empty gap requirement:

$$\boxed{c(d, g_i) = \frac{|\tau(d) \cap \tau(g_i)|}{|\tau(g_i)|}} \tag{4}$$

This is normalized token recall. The weighted coverage score is:

$$\boxed{C(d, G_t) = \frac{\sum_{i=1}^{m} w_i \, c(d, g_i)}{\sum_{i=1}^{m} w_i}} \tag{5}$$

Thus $C(d, G_t) \in [0, 1]$.

If $\tau(g_i)$ is empty, that invalid requirement is removed before retrieval.

4.7 Evidence Novelty

For non-empty evidence:

$$\boxed{N(d, E_t) = 1 - \max_{e \in E_t} s(d, e)} \tag{6}$$

A candidate very similar to existing evidence receives low novelty. A candidate dissimilar to all existing evidence receives high novelty.

At the first iteration:

$$E_0 = \emptyset.$$

We define:

$$\boxed{N(d, \emptyset) = 1.} \tag{7}$$

4.8 Final GAR Score

The final score is:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q, G_t) + \beta C(d, G_t) + \gamma N(d, E_t)} \tag{8}$$

subject to:

$$\boxed{\alpha, \beta, \gamma \geq 0, \qquad \alpha + \beta + \gamma = 1.} \tag{9}$$

Because the three components are normalized, the final score is also in $[0, 1]$.

4.9 Hyperparameters

GAR has four scoring hyperparameters:

$$\boxed{\alpha, \beta, \gamma, \lambda}$$

and one retrieval hyperparameter:

$$\boxed{K_0}.$$

Their roles are:

Parameter	**Role**	**Constraint**
α	Weight of relevance R	≥ 0
β	Weight of coverage C	≥ 0
γ	Weight of novelty N	≥ 0
λ	Question-vs-gap relevance balance	$[0, 1]$
K_0	ANN candidate-pool size	$K_0 \geq k$

The final retrieval depth k is an **experimental setting**, not a GAR-specific scoring parameter. It should be kept the same across systems for fair comparison.

The fixed embedding model and cosine similarity are also configuration choices, not learned GAR hyperparameters.

5 How GAR Works Using MIGRES

5.1 Overall Architecture

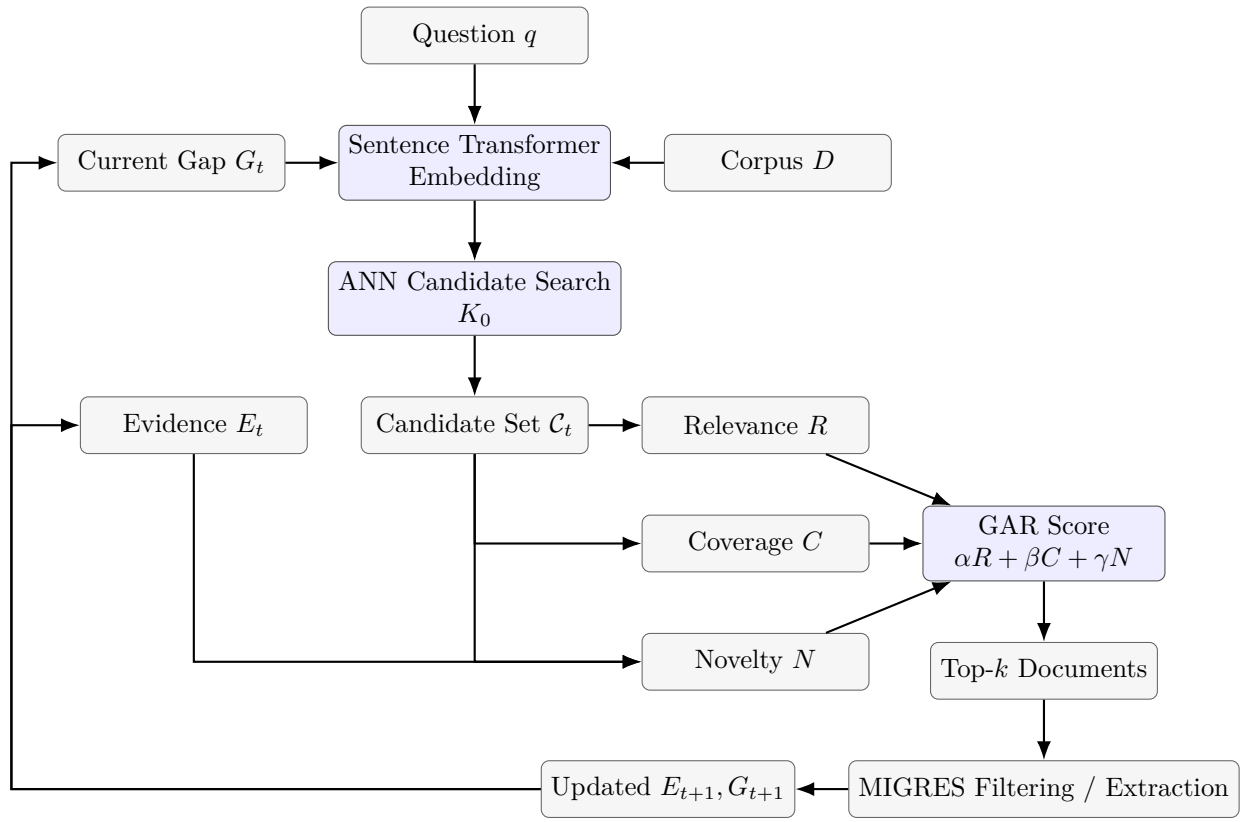


Figure 1: Overall architecture of GAR within the MIGRES framework. MIGRES provides the current question, evidence, and information gap. GAR generates candidates using ANN and ranks them using relevance, gap coverage, and evidence novelty. The retrieved documents are then returned to MIGRES for filtering, extraction, and state updating.

5.2 Step 1: Dataset Question

A benchmark supplies the original question:

$$q.$$

The benchmark's gold answer and supporting evidence are not given to GAR during retrieval.

5.3 Step 2: MIGRES Creates the Retrieval State

MIGRES processes the question and maintains:

$$E_t = \text{evidence already collected}$$

and:

$$G_t = \text{information still missing.}$$

The gap can contain multiple requirements:

$$G_t = \{g_1, \ldots, g_m\}.$$

GAR does not independently invent G_t in the main experiment. It receives the gap produced by MIGRES.

5.4 Step 3: Convert Text to Embeddings

GAR uses the fixed Sentence Transformer model:

$$\phi(x).$$

The question, each gap requirement, each document, and each evidence item can therefore be represented as vectors.
The document vectors are computed once before retrieval.

5.5 Step 4: ANN Candidate Search

ANN means **Approximate Nearest Neighbor**; it is not an artificial neural network.
The purpose of ANN is speed. A large corpus may contain millions of document vectors. Computing the full GAR score for every document would be unnecessarily expensive.
For each query-like input x, ANN returns K_0 nearby documents:

$$ANN_{K_0}(x, D).$$

GAR searches using both the original question and every gap requirement:

$$\boxed{\mathcal{C}_t = ANN_{K_0}(q, D) \cup \bigcup_{i=1}^{m} ANN_{K_0}(g_i, D)} \tag{10}$$

Duplicates are removed.
Thus K_0 is the candidate-pool size, while k is the final number returned.

5.6 Step 5: Calculate Relevance

For every candidate:

$$R(d, q, G_t)$$

measures both:

question-document relevance

and:

gap-document relevance.

5.7 Step 6: Calculate Coverage

GAR then asks:

"Does this document cover the information that is missing?"

This produces:

$$C(d, G_t).$$

5.8 Step 7: Calculate Novelty

GAR then asks:

"Is this document redundant with evidence already collected?"

This produces:

$$N(d, E_t).$$

5.9 Step 8: Calculate the GAR Score

For every candidate:

$$S(d \mid q, E_t, G_t) = \alpha R + \beta C + \gamma N.$$

The candidate with the highest score is preferred because it is simultaneously relevant, gap-covering, and non-redundant.

5.10 Step 9: Return Top-k

GAR returns:

$$\boxed{T_t = \mathrm{TopK}_{d \in \mathcal{C}_t} S(d \mid q, E_t, G_t)} \tag{11}$$

where k is fixed for the experiment.

5.11 Step 10: MIGRES Continues

MIGRES receives the retrieved documents and performs its normal filtering and extraction.
It updates:

$$E_{t+1}$$

and, if more information is needed:

$$G_{t+1}.$$

GAR is then called again.
The complete loop is:

$$\boxed{(q, E_t, G_t, D) \xrightarrow{\text{GAR}} T_t \xrightarrow{\text{MIGRES}} (E_{t+1}, G_{t+1}) \xrightarrow{\text{GAR}} T_{t+1}}$$

This is why GAR is evaluated inside MIGRES rather than as an isolated one-shot retriever.

5.12 Candidate Generation vs. GAR Ranking

The two stages must not be confused:

$$\boxed{\text{Embedding} \rightarrow \text{ANN candidate generation} \rightarrow \text{GAR scoring} \rightarrow \text{Top-}k}$$

Embedding creates vectors.
ANN finds a manageable candidate set.
GAR makes the final ranking decision.
If BM25 is used to create the candidate pool and GAR only reranks those documents, that is a *GAR reranker* experiment, not the complete GAR retrieval experiment proposed here.

5.13 Development, Validation, and Test Protocol

5.13.1 Is GAR Trained?

No.
The proposed GAR algorithm has no trainable parameters. We do not train GAR on D_{train}.
The embedding model is a fixed pretrained Sentence Transformer. It is not fine-tuned in the primary experiment.
Therefore the experiment has no GAR training stage.

5.13.2 Training Data

If a baseline uses an official pretrained checkpoint, we use that checkpoint rather than retraining it on the evaluation data.
The benchmark training split may be used for auxiliary development or baseline-specific training only if required by the official implementation. Any such use must be documented and must not use test annotations.
For GAR itself:

$$D_{\text{train}} \not\to \text{GAR parameter learning.}$$

5.13.3 Validation Set D_{val}

We define:

$$\boxed{D_{\text{val}} = \text{the held-out development/validation split.}}$$

Where an official development split exists, we use it. The exact files and split names are recorded for each dataset.
GAR hyperparameters are selected using the *complete MIGRES+GAR pipeline* on D_{val}.
This is important because GAR cannot be meaningfully validated in isolation: E_t and G_t are produced by the MIGRES iterative process.
The validation loop is:

$$D_{\text{val}} \to \text{MIGRES} \to (q, E_t, G_t) \to \text{GAR} \to \text{MIGRES} \to \text{validation metric.}$$

5.13.4 Bayesian Optimization

We use Bayesian optimization to select:

$$\theta_{\text{GAR}} = \{\alpha, \beta, \gamma, \lambda, K_0\}.$$

Bayesian optimization uses previous validation evaluations to decide which configuration to test next rather than exhaustively evaluating every possible combination [Snoek et al., 2012].
The search space is:

$$\alpha, \beta, \gamma \geq 0, \qquad \alpha + \beta + \gamma = 1,$$

$$0 \leq \lambda \leq 1,$$

and a predefined discrete candidate set for K_0, for example:

$$K_0 \in \{50, 100, 200, 500\}.$$

The exact search space and random seed are recorded before the experiment.
The optimization objective must be selected before test evaluation. The primary objective should be a retrieval metric aligned with the main research question, such as Recall@k on D_{val}.
The procedure is:

$$\boxed{D_{\text{val}} \to \text{MIGRES+GAR} \to \text{validation score} \to \text{Bayesian optimization} \to \theta^*_{\text{GAR}}}$$

The selected configuration is then frozen.

5.13.5 Test Set D_{test}

After validation:

$$\boxed{\theta^*_{\text{GAR}} \text{ is frozen.}}$$

We then evaluate:

$$D_{\text{test}} \rightarrow \text{MIGRES+GAR}(\theta^*_{\text{GAR}}) \rightarrow \text{final metrics.}$$

No test result is used to change the hyperparameters.
The same k, dataset, corpus, MIGRES configuration, and stopping conditions are used for the comparison systems wherever possible.

5.13.6 Complete Experimental Flow

1. Prepare the official dataset splits and searchable corpus.
2. Reproduce MIGRES+BM25.
3. Verify the MIGRES retrieval interface and confirm how E_t and G_t are exposed.
4. Build the fixed GAR document embeddings.
5. Build the ANN index.
6. Implement and unit-test the GAR equations.
7. Run Bayesian optimization on D_{val} through the complete MIGRES+GAR pipeline.
8. Freeze $\alpha, \beta, \gamma, \lambda, K_0$.
9. Run the held-out D_{test} with the frozen configuration.
10. Run BM25, DPR, Contriever, ColBERTv2, and SPLADE using their established implementations.
11. Evaluate retrieval quality, downstream answer quality, efficiency, and cost.
12. Run GAR ablations using the same frozen evaluation protocol.

6 Datasets

6.1 Dataset Selection

The project uses the six datasets reported in the MIGRES evaluation:

$$\boxed{\text{WikiHop, HotpotQA, MuSiQue, Natural Questions, TriviaQA, StrategyQA}}$$

These datasets provide questions and task-specific answer/evidence annotations suitable for evaluating retrieval and downstream question answering.

6.2 WikiHop

Paper: Welbl et al. [2018].
WikiHop is a multi-document, multi-hop reading-comprehension benchmark designed so that the answer requires combining information across documents. The dataset is useful for GAR because it tests whether retrieval can assemble complementary evidence rather than repeatedly retrieving one topic.
Typical fields include a question, candidate documents, an answer, and supporting information.
Paper: https://aclanthology.org/Q18-1021/

6.3 HotpotQA

Paper: Yang et al. [2018].
HotpotQA contains about 113K Wikipedia-based question-answer pairs and provides sentence-level supporting facts. It includes multi-hop and comparison questions [Yang et al., 2018].
For first-stage development, we use the FullWiki setting because it requires retrieval from a larger Wikipedia corpus. The official repository provides train, development, and FullWiki resources and explicitly allows researchers to replace its retrieval component with their own IR system.
Paper: https://aclanthology.org/D18-1259/
Official resources: https://hotpotqa.github.io/
The repository documentation describes the FullWiki development and test files and the use of custom IR systems.

6.4 MuSiQue

Paper: Trivedi et al. [2022].
MuSiQue was designed to reduce shortcut solutions in multi-hop QA. MuSiQue-Ans contains about 25K questions requiring 2–4 hops [Trivedi et al., 2022].
Typical information includes a question, answer, reasoning structure, and supporting evidence.
Paper: https://aclanthology.org/2022.tacl-1.31/
Official repository: https://github.com/StonyBrookNLP/musique

6.5 Natural Questions

Paper: Kwiatkowski et al. [2019].
Natural Questions consists of real anonymized queries issued to Google Search and Wikipedia-based annotations. The public release contains 307,373 training examples, 7,830 development examples, and 7,842 sequestered test examples [Kwiatkowski et al., 2019].
Typical fields include a question, Wikipedia page information, long-answer annotations, short-answer annotations, and document-level information.
Paper: https://aclanthology.org/Q19-1026/
Official research page: https://ai.google/research/NaturalQuestions

6.6 TriviaQA

Paper: Joshi et al. [2017].
TriviaQA contains about 95K question-answer pairs and more than 650K question-answer-evidence triples. Questions were written by trivia enthusiasts and paired with independently gathered evidence documents [Joshi et al., 2017].
Typical fields include a question, answer, aliases, and evidence documents.
Paper: https://aclanthology.org/P17-1147/
Official resources: https://nlp.cs.washington.edu/triviaqa/

6.7 StrategyQA

Paper: Geva et al. [2021].
StrategyQA contains 2,780 questions with implicit reasoning strategies. Each example includes a question, a reasoning decomposition, and Wikipedia evidence paragraphs [Geva et al., 2021].
This makes it useful for testing whether gap-aware retrieval helps when the required reasoning steps are not directly stated in the question.
Paper: https://aclanthology.org/2021.tacl-1.21/
Official resources: https://allenai.org/data/strategyqa

6.8 Dataset Summary

Dataset	Scale	Useful properties for GAR
WikiHop	∼51K	Multi-document, multi-hop reasoning.
HotpotQA	∼113K	Multi-hop QA with sentence-level supporting facts; FullWiki retrieval.
MuSiQue	∼25K	2–4 hop compositional reasoning.
Natural Questions	307,373 train; 7,830 dev; 7,842 test	Real search questions with Wikipedia annotations.
TriviaQA	∼95K QA pairs; >650K evidence triples	Heterogeneous evidence and lexical variation.
StrategyQA	2,780	Implicit reasoning with evidence paragraphs.

Exact counts used in experiments must be taken from the downloaded release and recorded in the experiment configuration. Dataset counts can differ by release or preprocessing.

6.9 What a Dataset Example Looks Like

The common conceptual structure is:

$$\boxed{\text{Question} + \text{Corpus/Evidence} + \text{Answer} + \text{Ground-Truth Evidence/Annotations}}$$

For example, a HotpotQA-style example contains a natural-language question, an answer, supporting facts, and Wikipedia context. During retrieval, GAR receives the question and searchable corpus through MIGRES. The gold answer and gold supporting evidence are reserved for evaluation.
The ground-truth evidence must never be used to construct G_t, E_t, or the ANN candidate pool during test-time retrieval.

6.10 Dataset Split Policy

For each dataset:

$$D_{\text{train}}, \quad D_{\text{val}}, \quad D_{\text{test}}$$

are treated separately where official splits are available.
For GAR:

$$D_{\text{val}}$$

is used for hyperparameter optimization, while:

$$D_{\text{test}}$$

is used only for the final evaluation.
If an official public test set has hidden answers, its evaluation follows the dataset's official evaluation procedure.

7 Experiments

7.1 Research Questions

RQ1. Does GAR improve retrieval quality over the MIGRES BM25 baseline?

$$\boxed{\text{MIGRES+GAR} > \text{MIGRES+BM25?}}$$

RQ2. Does GAR outperform strong dense, sparse, and late-interaction retrieval methods?

$$\text{GAR} \quad \text{vs.} \quad \{\text{DPR, Contriever, ColBERTv2, SPLADE}\}.$$

RQ3. Does replacing BM25 with GAR improve downstream MIGRES answer quality?
RQ4. Which GAR component contributes most: relevance, coverage, or novelty?
RQ5. Does GAR generalize across the six datasets?
RQ6. What is the computational cost of GAR compared with the baselines?
RQ7. Does GAR improve retrieval without adding unnecessary LLM calls?

7.2 Hypotheses

Primary hypothesis:

$$H_0: \quad \text{GAR does not improve retrieval performance over BM25.}$$

$$H_1: \quad \text{GAR improves retrieval performance over BM25.}$$

Secondary hypothesis:

$$H_0: \quad \text{GAR} \leq \text{best baseline.}$$

$$H_1: \quad \text{GAR} > \text{best baseline.}$$

For ablations:

$$H_0: \quad \text{removing a GAR component does not change performance.}$$

$$H_1: \quad \text{removing a GAR component reduces performance.}$$

We report effect sizes and confidence intervals together with statistical significance. The exact paired statistical test should be fixed before inspecting the final test results; a paired bootstrap or paired permutation procedure is suitable for per-question retrieval metrics.

7.3 Main Comparison

The main experiment compares:

MIGRES+BM25
MIGRES+DPR
MIGRES+Contriever
MIGRES+ColBERTv2
MIGRES+SPLADE
MIGRES+GAR

The same questions, corpus, k, MIGRES settings, answer-generation model, and stopping conditions should be used wherever possible.

7.4 Ablation Studies

The full GAR model is:

$$S_{\text{GAR}} = \alpha R + \beta C + \gamma N.$$

We remove one component at a time.
Without relevance:

$$\boxed{S_{-R}(d) = \beta C(d, G_t) + \gamma N(d, E_t)}$$

Without coverage:

$$\boxed{S_{-C}(d) = \alpha R(d, q, G_t) + \gamma N(d, E_t)}$$

Without novelty:

$$\boxed{S_{-N}(d) = \alpha R(d, q, G_t) + \beta C(d, G_t)}$$

We also recommend two additional diagnostic ablations:

$$GAR_{\text{Q-only}}$$

which uses only question relevance, and:

$$GAR_{\text{Q+G}}$$

which uses question and gap relevance but not novelty.
These help determine whether the main benefit comes from simply adding the gap representation or from the complete relevance–coverage–novelty combination.

7.5 Candidate-Pool Sensitivity

Because K_0 controls how many candidates reach the full GAR scorer, we test:

$$K_0 \in \{50, 100, 200, 500\}$$

on the validation set during optimization.
The test experiment uses only the selected value.
We report both retrieval quality and latency as K_0 changes.

7.6 Retrieval Metrics

7.6.1 Recall@k

Recall@k measures how much of the ground-truth relevant evidence appears in the first k retrieved documents:

$$\boxed{Recall@k = \frac{|\text{relevant evidence} \cap \text{top-}k|}{|\text{relevant evidence}|}}$$

This is a primary metric because GAR is designed to retrieve missing evidence.

7.6.2 Precision@k

$$\boxed{Precision@k = \frac{|\text{relevant documents in top-}k|}{k}}$$

Precision measures how much of the retrieved set is relevant.

7.6.3 MRR

Mean Reciprocal Rank measures how early the first relevant result appears:

$$\boxed{MRR = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{rank_i}}$$

where $rank_i$ is the rank of the first relevant result for question i.

7.6.4 nDCG@k

For graded relevance:

$$DCG@k = \sum_{i=1}^{k} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

and:

$$\boxed{nDCG@k = \frac{DCG@k}{IDCG@k}}$$

where $IDCG@k$ is the ideal DCG at cutoff k [Järvelin and Kekäläinen, 2002].

7.6.5 Evidence Coverage

Because GAR explicitly models an information gap, we additionally report an evidence-coverage measure where the dataset annotations permit it:

$$\boxed{GapEvidenceCoverage@k = \frac{\text{ground-truth evidence requirements retrieved in top-}k}{\text{total ground-truth evidence requirements}}}$$

The exact mapping from benchmark annotations to evidence requirements must be defined separately for each dataset.

7.7 Downstream QA Metrics

The final MIGRES answer is evaluated using the metrics supported by each benchmark.
These may include:

- Exact Match (EM);
- token-level F1;
- answer accuracy for classification-style datasets;
- evidence/supporting-fact quality where annotations permit.

For a benchmark with a normalized answer set A_i:

$$EM_i = \mathbb{1}[\hat{a}_i \in A_i].$$

Average exact match is:

$$EM = \frac{1}{N} \sum_{i=1}^{N} EM_i.$$

For token-level F1, precision and recall are computed over normalized answer tokens and then combined as:

$$F1 = \frac{2PR}{P+R}.$$

The benchmark's official evaluation script should be used whenever available.

7.8 Efficiency Metrics

We report:

- retrieval latency per question;
- ANN indexing time;
- index memory;
- number of retrieval iterations;
- number of documents scored by GAR;
- number of LLM calls in the complete MIGRES system;
- input and output tokens;
- API cost where applicable.

The GAR scorer itself should not require one LLM call per candidate. Embeddings, cosine similarity, lexical coverage, and novelty can be calculated locally.

7.9 API Cost

If an external model is used by MIGRES, the cost for question q is:

$$\boxed{Cost(q) = T_{\text{in}}(q)p_{\text{in}} + T_{\text{out}}(q)p_{\text{out}}}$$

where T_{in} and T_{out} are input and output token counts and $p_{\text{in}}, p_{\text{out}}$ are the corresponding prices. Average cost is:

$$\boxed{\overline{Cost} = \frac{1}{N}\sum_{i=1}^{N} Cost(q_i)}$$

The comparison must distinguish GAR's local retrieval computation from the LLM cost of the surrounding MIGRES framework.

7.10 Fairness Controls

The following should remain fixed across systems:

- dataset split;
- question set;
- document corpus;
- final retrieval depth k;
- MIGRES components outside retrieval;
- answer-generation model;
- stopping conditions;
- evaluation scripts;
- random seeds where applicable.

Only the retrieval method should change.

7.11 Reproducibility Configuration

The final experiment record should contain:

Parameter	Value
Dataset	Exact release and split
Corpus	Exact corpus version
Retriever	BM25 / DPR / Contriever / ColBERTv2 / SPLADE / GAR
Baseline checkpoint	Exact model/checkpoint
GAR embedding model	`multi-qa-mpnet-base-cos-v1`
Final retrieval depth k	Fixed value for all systems
ANN candidate pool K_0	Validation-selected GAR value
α	Validation-selected value
β	Validation-selected value
γ	Validation-selected value
λ	Validation-selected value
Bayesian optimization seed	Recorded value
ANN implementation	Exact library/index
Hardware	CPU/GPU configuration
Software	Python and package versions
MIGRES configuration	Exact prompts/model/settings

8 Conclusion

This project proposes Gap-Aware Retrieval (GAR) as a retrieval algorithm for iterative information-seeking systems. GAR is evaluated inside MIGRES rather than replacing the MIGRES framework itself.
The core idea is:

$$\boxed{\text{Relevant} + \text{Gap-covering} + \text{Non-redundant}}$$

implemented as:

$$\boxed{S(d \mid q, E_t, G_t) = \alpha R(d, q, G_t) + \beta C(d, G_t) + \gamma N(d, E_t)}$$

The complete retrieval pipeline is:

$$\boxed{\text{Sentence Transformer} \rightarrow \text{Cosine Similarity} \rightarrow \text{ANN Candidate Search} \rightarrow \text{GAR Scoring} \rightarrow \text{Top-}k \rightarrow \text{MIGRES}}$$

GAR has no training stage in the primary experiment. Its scoring hyperparameters are selected on D_{val} through the complete MIGRES+GAR pipeline using Bayesian optimization and then frozen for D_{test}.
The central empirical question is whether using the information gap and previously collected evidence produces more useful retrieval than standard retrieval algorithms.

References

Keheng Wang, Feiyu Duan, Peiguang Li, Sirui Wang, and Xunliang Cai. LLMs know what they need: Leveraging a missing information guided framework to empower retrieval-augmented generation. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 2379–2400. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.coling-main.163/.

Vikas Yadav, Steven Bethard, and Mihai Surdeanu. Unsupervised alignment-based iterative evidence retrieval for multi-hop question answering. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4514–4525. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.414. URL https://aclanthology.org/2020.acl-main.414/.

Marc Pickett, Jeremy Hartman, Ayan Kumar Bhowmick, Raquib-ul Alam, and Aditya Vempaty. Better rag using relevant information gain. *arXiv preprint arXiv:2407.12101*, 2024. URL https://arxiv.org/abs/2407.12101.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 3982–3992. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1410. URL https://aclanthology.org/D19-1410/.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *arXiv preprint arXiv:1702.08734*, 2017. URL https://arxiv.org/abs/1702.08734.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019. URL https://doi.org/10.1561/1500000019.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.550. URL https://aclanthology.org/2020.emnlp-main.550/.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 2665–2679. Association for Computational Linguistics, 2022. URL https://aclanthology.org/2022.emnlp-main.738/.

Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.naacl-main.272. URL https://aclanthology.org/2022.naacl-main.272/.

Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for first stage ranking. *arXiv preprint arXiv:2107.05720*, 2021. URL https://arxiv.org/abs/2107.05720.

Sentence Transformers. multi-qa-mpnet-base-cos-v1. Hugging Face model card, 2020. URL https://huggingface.co/sentence-transformers/multi-qa-mpnet-base-cos-v1. 768-dimensional sentence-transformers model designed for semantic search.

Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25, 2012. URL https://arxiv.org/abs/1206.2944.

Johannes Welbl, Pontus Stenetorp, and Sebastian Riedel. Constructing datasets for multi-hop reading comprehension across documents. *Transactions of the Association for Computational Linguistics*, 6:287–302, 2018. doi: 10.1162/tacl_a_00021. URL https://aclanthology.org/Q18-1021/.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380. Association for Computational Linguistics, 2018. doi: 10.18653/v1/D18-1259. URL https://aclanthology.org/D18-1259/.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022. doi: 10.1162/tacl_a_00475. URL https://aclanthology.org/2022.tacl-1.31/.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://aclanthology.org/Q19-1026/.

Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1601–1611. Association for Computational Linguistics, 2017. doi: 10.18653/v1/P17-1147. URL https://aclanthology.org/P17-1147/.

Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 9:346–361, 2021. doi: 10.1162/tacl_a_00370. URL https://aclanthology.org/2021.tacl-1.21/.

Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002. doi: 10.1145/582415.582418. URL https://doi.org/10.1145/582415.582418.